

PIMoG: An Effective Screen-shooting Noise-Layer Simulation for Deep-Learning-Based Watermarking Network

Han Fang
fanghan@nus.edu.sg
National University of Singapore

Zhaoyang Jia
jzy_ustc@mail.ustc.edu.cn
University of Science and Technology
of China

Zehua Ma
mzh045@mail.ustc.edu.cn
University of Science and Technology
of China

Ee-Chien Chang*
changec@comp.nus.edu.sg
National University of Singapore

Weiming Zhang*
zhangwm@ustc.edu.cn
University of Science and Technology
of China

ABSTRACT

With the omnipresence of camera phone and digital display, capturing digitally displayed image with camera phone are getting widely practiced. In the context of watermarking, this brings forth the issue of screen-shooting robustness. The key to acquiring screen-shooting robustness is designing a good noise layer that could represent screen-shooting distortions in a deep-learning-based watermarking framework. However, it is very difficult to quantitatively formulate the screen-shooting distortion since the screen-shooting process is too complex. In order to design an effective noise layer for screen-shooting robustness, we propose new insight in this paper, that is, it is not necessary to quantitatively simulate the overall procedure in the screen-shooting noise layer, only including the most influenced distortions is enough to generate an effective noise layer with strong robustness. To verify this insight, we propose a screen-shooting noise layer dubbed PIMoG. Specifically, we summarize the most influenced distortions of screen-shooting process into three parts (perspective distortion, illumination distortion and moiré distortion) and further simulate them in a differentiable way. For the rest distortion, we utilize the Gaussian noise to approximate the main part of them. As a result, the whole network can be trained end-to-end with such noise layer. Extensive experiments illustrate the superior performance of the proposed PIMoG noise layer. In addition to the noise layer design, we also propose a gradient mask-guided image loss and an edge mask-guided image loss to further improve the robustness and invisibility of the whole network respectively. Based on the proposed loss and PIMoG noise layer, the whole framework outperforms the SOTA watermarking method with at least 5% in extraction accuracy and achieves more than 97% accuracy in different screen-shooting conditions.

CCS CONCEPTS

• **Security and privacy** → *Domain-specific security and privacy architectures; Digital rights management.*

*Corresponding Author.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike International 4.0 License.

MM '22, October 10–14, 2022, Lisboa, Portugal
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9203-7/22/10.
<https://doi.org/10.1145/3503161.3548049>

KEYWORDS

watermarking, screen-shooting, noise-layer simulation

ACM Reference Format:

Han Fang, Zhaoyang Jia, Zehua Ma, Ee-Chien Chang, and Weiming Zhang. 2022. PIMoG: An Effective Screen-shooting Noise-Layer Simulation for Deep-Learning-Based Watermarking Network. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, October 10–14, 2022, Lisboa, Portugal. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3503161.3548049>

1 INTRODUCTION

Digital watermarking technology can serve as a way to realize leak source tracing. By embedding unique identification information with different copies, the leaked one can be easily identified. The most important property of watermarking scheme is robustness. Since during transmission, the leaked copy may suffer a lot of distortions, the key to achieving traceability is that the watermark should be robust to the distortions. Traditional digital watermarking pays more attention to the robustness of digital editing distortion [1, 3, 5, 11–13, 19, 26, 29], such as JPEG compression, Gaussian noising and so on, because the traditional information leakage mostly occurs in electronic channel. However, with the development of digital devices, one can easily get a high-quality copy by photographing with mobile phones, and screen-shooting process may seriously damage the watermark signal but get little impact on digital files, as shown in Fig. 1. Therefore, designing a screen-shooting robust watermarking framework has become an urgent requirement at this stage.

Traditional digital watermarking schemes often design the algorithm based on the analysis of the distortion. For example, in order to resist JPEG distortion, some algorithms will choose to embed the watermark in DCT domain [16]. However, due to the complexity of the screen-shooting process, it is difficult to quantitatively analyze the screen-shooting distortion. Therefore, it is a challenge to design an algorithm to ensure the robustness of the screen-shooting process. For traditional screen-shooting resilient watermarking algorithms, they only qualitatively analyze the possible distortion in the screen-shooting process and use some general watermark embedding methods to ensure robustness [6]. However, such kinds of methods often produces serious visual distortions, because the embedding method is non-adaptive and cannot make full use of host image features.

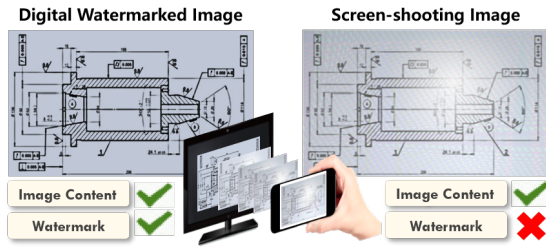


Figure 1: The screen-shooting process as well as its influence.

To make full use of image features and design better algorithms, deep-learning-based watermarking architectures are proposed [2, 25, 27, 28]. The backbone of deep-learning-based watermarking is an end-to-end auto-encoder-like architecture, which contains an encoder, a decoder and a noise layer. The key to ensuring robustness is designing a good noise layer. By training with the targeted noise layer, the corresponding robustness can be guaranteed. But the premise to be trainable is that the noise layer must be differentiable, so when facing screen-shooting distortion which is a non-differentiable distortion, such framework cannot be directly applied.

An intuitive idea for non-differentiable distortions is to approximate the whole distortion process with a neural network and serve it as the noise layer for training. But such method needs a lot of data, and generating training data is a time-consuming and labor-consuming process. Besides, training with the simulated neural network may arise the mismatch problem between the training data and the testing data if it comes from different devices, which will further influence the generalization of the algorithm.

Therefore, the biggest challenge in realizing screen-shooting robustness is designing an effective screen-shooting noise layer. To achieve this goal, in this paper, we propose a new insight: It is not necessary to simulate the overall procedure in the screen-shooting noise layer, only including the most influenced distortions is enough to generate an effective noise layer with strong robustness.

To justify the correctness of the insight, we generate a screen-shooting noise layer dubbed PIMoG. Specifically, we summarize the most influenced distortions that occurred in screen-shooting process into three main parts: perspective distortion, illumination distortion and moiré distortion. And for each distortion, we propose a differentiable formulation to approximate it. The whole noise layer consists of the simulated perspective distortion, the simulated illumination distortion, the simulated moiré distortion and a Gaussian noise layer that is used for representing the main part of the rest distortions. Extensive experiments have proved the correctness of our insight and the effectiveness of the designed noise layer. Compared with network simulation, the proposed PIMoG noise layer has obvious advantages in robustness and adaptability.

Besides the noise layer, we also design a gradient mask-guided image loss to further improve the robustness and an edge mask-guided image loss for better invisibility. Training with PIMoG and these two losses, the proposed watermarking network can both achieve high invisibility and strong robustness.

The contribution of the proposed schemes are shown as follows:

- 1). We give new insight in designing the screen-shooting noise layer that it is not necessary to simulate the overall screen-shooting process, only formulating the most influenced distortions in such process as the noise layer is enough to guarantee strong robustness. We hope this idea may inspire the follow-up work of designing complex noise layers in deep-learning-based watermarking framework.

- 2). We propose a noise layer named PIMoG which contains the most influenced distortions in screen-shooting process and we further formulate these distortions in a differentiable way. By training with PIMoG, the screen-shooting robustness can be well achieved.

- 3). We design a gradient mask-guided image loss which improves the screen-shooting robustness and an edge mask-guided image loss which helps the encoder embedding watermark in a high-quality way.

- 4). Various experimental results indicate the outstanding performance of the proposed scheme against the screen-shooting distortions on the premise of high visual quality.

The remaining of this paper is organized as follows. In Section 2, we mainly discuss the related work of the proposed scheme. Section 3 introduces the architecture of the proposed watermarking scheme. The corresponding experimental results are indicated in Section 4. Section 5 introduces the potential applications of the proposed noise layer. And Section 6 concludes the paper.

2 RELATED WORK

2.1 Traditional watermarking schemes

Traditional watermarking schemes are extensively researched, but most of them are still working on image processing robustness. Recently, several works paid attention to the camera shooting distortions. Nakamura *et al.* [20] first proposed to use two orthogonal templates to represent watermark signals and embed them in a superimposed way. After that Pramila *et al.* [21] followed such works and proposed several works for better performance. However, such schemes can only use for print-shooting process. Gugelmann *et al.* [7] suggested embedding the watermark into the screen by modulating the luminance of different regions. But such scheme cannot well balance the robustness and invisibility. Fang *et al.* [6] analyzed the screen-shooting distortion and proposed a SIFT and DCT based method to achieve screen-shooting robustness. But such algorithm will arise visual distortion in embedding region. Since traditional watermarking schemes only use handcrafted features for embedding and extracting, they cannot make full use of the characteristics of the host image.

2.2 Deep learning based watermarking schemes

Recently, more deep-learning-based architectures for watermarking schemes are proposed. Tancik *et al.* [22] trained the network with several differentiable operations to acquire print-shooting robustness. But such noise layer is trying to simulate the overall process of print-shooting process which not only cannot be applied to screen-shooting scenarios but also will arise serious visual distortion. Recently, Wengrowski *et al.* [24] trained a camera-display transfer function (CDTF) network to simulate the camera shooting process. By setting CDTF as the noise layer, the proposed network

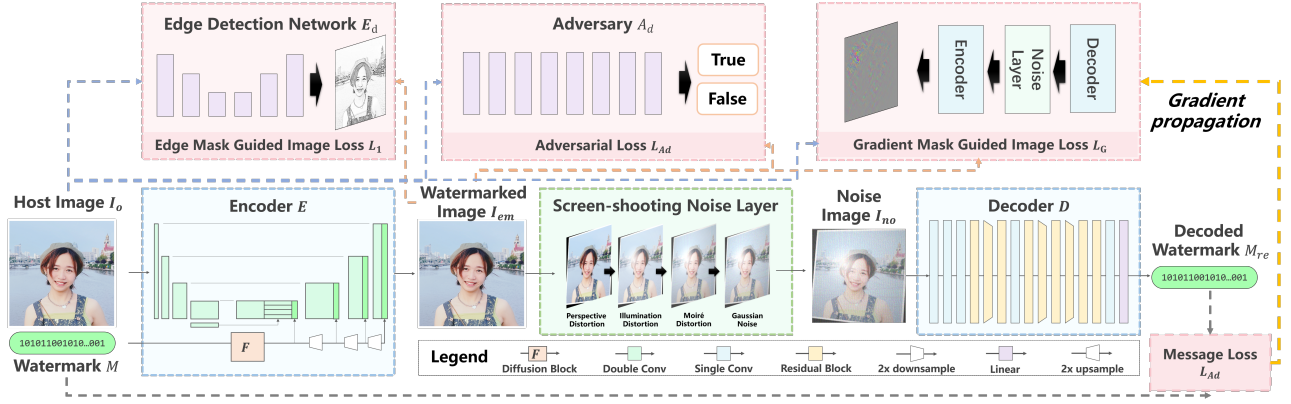


Figure 2: The framework of the whole system. It consist of three main parts: the encoder, the screen-shooting noise layer and the decoder. The whole network can be trained end-to-end. And in order to both achieve screen-shooting robustness and high visual quality, we proposed the edge mask-guided image loss as well as the gradient mask-guided image loss to cope with the whole framework.

can realize the screen-shooting resilience. However, a large number of screen-camera image pairs are needed to train CDTF, and such scheme got poor performance on data generated with new devices. Liu *et al.* [18] proposed a two-stage training strategy for non-differentiable distortions. In stage-1, they first initialized a noise-free encoder and decoder, and in stage-2, they used the distorted image to further fine-tune the decoder to be adaptive to the distortions. However, since the encoded feature may be erased after screen-shooting process, even after fine-tuning, robustness still cannot be effectively guaranteed.

3 PROPOSED FRAMEWORK

In order to realize the robustness of screen-shooting process, we propose a screen-shooting noise layer dubbed PIMoG and a corresponded framework, as shown in Fig. 2. The whole framework contains seven main parts: (1) The diffusion block F , which receives watermark message $M \in \{0, 1\}$ with length L as input and outputs the message hidden layer F_m . F_m will be further reshaped and concatenated to the hidden layer of the encoder; (2) the encoder E with parameters θ_E , which is fed with the host image $I_o \in \mathbb{R}^{C \times H \times W}$ and F_m to generate the watermarked image $I_{em} \in \mathbb{R}^{C \times H \times W}$; (3) the pre-trained edge mask generation network E_d , which aims to generate the edge mask E_m of I_o ; (4) the adversary A_d with parameters θ_{A_d} , which tries to judge whether I_{em} is a watermarked image or not; (5) the screen-shooting noise layer N_o which consist of several differentiable distortions and aims to distort I_{em} to generate the noised image I_{no} ; (6) the decoder D with parameter θ_D , which receives I_{no} as input and output the decoded message M_{de} ; (7) the gradient mask generator, which aims to calculate the gradient mask G_m of message loss in the host image via loss propagation. It is worth noting that the edge mask generation network E_d is pre-trained and fixed, which means it will not take participate in the final watermark embedding process after training.

3.1 Diffusion block

In order to better couple message and image features and embed messages into host images, we propose the diffusion block F to pre-process the message. F is a full connection layer that receives the message $M \in \{0, 1\}$ of length L and output a 256 length vector,

and such vector is further reshaped and repeated to generate the message hidden layer $F_m \in \mathbb{R}^{64 \times 16 \times 16}$. Then F_m will be up-sampled to the same size of the encoder's hidden layers respectively, as shown in Fig. 2.

3.2 Encoder

The structure of the encoder E is shown in Fig. 2. The encoder aims to embed the watermark into the host image with as little visual distortion as possible.

Specifically, three “double-conv” blocks ($2 \times \text{conv-bn-relu-maxpool}$) first downsample I_o to $H/8 \times W/8$ feature maps. Then an extra convolutional layer is applied on the feature maps and generates a global $H/32 \times W/32$ feature maps, which is further concatenated to the $H/8 \times W/8$ feature maps. Finally, several “up-double-conv” blocks ($2 \times \text{upsampling-conv-bn-relu-maxpool}$) layers are adopted to generate the final watermarked image I_{em} which has the same size as the original image. Among them, the message hidden layer F_m s after reshaping and up-sampling are concatenated to the hidden layers of E . We believe that concatenating the hidden layer of the host image and the watermark message is a very effective way to combine features and generate better images.

3.3 The pre-trained edge mask generator

Giving a large weight to the edge region of the image when embedding the watermark signal is an effective way to improve the visual quality of the watermarked image [15]. To automatically achieve this in deep-learning framework, we proposed a mask-guided image loss. The mask is generated by a pre-trained edge detection network BDCN [8] which is proved to be effective in calculating the edge of the image. So after generating the edge mask $E_m \in \{0, 1\}$, we can set the loss function of the encoded image as:

$$\mathcal{L}_I = E'_m * \text{MSE}(I_o, I_{em}) = E'_m * \text{MSE}(I_o, E(\theta_E, I_o, M)) \quad (1)$$

where

$$E'_m = E_m \times \alpha_E + \beta_E \quad (2)$$

α_E and β_E is used for adjusting the weights of the mask. In this paper, α_E is set as 1 and β_E is set as 1 by default. In E_m , the edge region maintains the lower value.



Figure 3: The simulated screen-shooting noise layer which consists of the perspective distortion, the illumination distortion and extra noise.

3.4 Adversary

To acquire better visual quality of watermarked images, an adversarial network is adopted in the framework. On one hand, the encoded image is trying to mislead the adversarial network, which will result in a similar image as the host image. So L_{Ad} loss is used to improve the image quality of I_{em} by updating θ_E :

$$\mathcal{L}_{Ad} = \log(Ad(I_{em})) = \log(Ad(E(\theta_E, I_o, M))) \quad (3)$$

Besides, Ad should update to give a correct classification between I_{em} and I_o , which is realized by updating θ_{Ad} with:

$$\mathcal{L}_{Ado} = \log(1 - Ad(\theta_{Ad}, I_{em})) = \log(1 - Ad(\theta_{Ad}, E(I_o, M))) \quad (4)$$

In this paper, we use PatchGAN [10] by default.

3.5 Screen-shooting noise layer

As analyzed before, we propose a novel insight that it is not necessary to simulate the overall process as the noise layer, instead, we only formulate the most influenced distortions and mix them as a combined noise layer for training. We believe the combined noise layer will lead to stronger robustness. In this section, we will introduce the noise layer in detail.

After the investigation of screen-shooting process, we conclude the most influenced distortions in three main parts: perspective distortion, illumination distortion and moiré distortion. Then we formulate each of them in a differentiable way. As for the main part of the rest distortion, we follow the settings in [22] and use the Gaussian noise as an approximation. So in general, we propose the PIMoG noise layer which contains the following several parts, as shown in Fig. 3.

3.5.1 Perspective distortion. Perspective distortion comes from the different shooting angles, which will result to different deformations in the captured image. In this paper, we generate a random homography mapping to simulate the effect of perspective distortions. Specifically, as shown in Fig. 4, the 4 vertices of the left image are denoted as $P_1(x_1, y_1)$, $P_2(x_2, y_2)$, $P_3(x_3, y_3)$ and $P_4(x_4, y_4)$. And the transformed coordinates corresponding to these 4 points in the right part of the image are denoted as $P'_1(x'_1, y'_1)$, $P'_2(x'_2, y'_2)$, $P'_3(x'_3, y'_3)$ and $P'_4(x'_4, y'_4)$. Substituting the 8 coordinates into Eq.(5) we can get 8 equations, so the value of $a_1, b_1, c_1, a_0, b_0, c_2$ can be obtained by solving these equations.

$$\begin{aligned} x' &= \frac{a_1x + b_1y + c_1}{a_0x + b_0y + 1} \\ y' &= \frac{a_2x + b_2y + c_2}{a_0x + b_0y + 1} \end{aligned} \quad (5)$$

After determining all the parameters, the perspective mapping is generated. And all the pixels can be mapped into a new coordinate

in the generated image according to the mapping. In this paper, we randomly perturb four corners of the images (up to ± 8 pixels) and apply them as the new coordinates of each corner. And then we bilinearly sample each pixel of the original watermarked image I_{em} into the perspective warped version to generate the perspective warped image I_{pD} .

3.5.2 Illumination distortion. In screen-shooting process, due to the influence of environment illumination and screen illumination, different shooting conditions will result in different illumination distributions in the captured image. We simulate the illumination conditions with two categories: point light source and line light source. In this paper, we propose to utilize point spread distribution and line spread distribution to approximate these conditions. Specifically, for point light source distortion, we generate the distribution weight matrix as:

$$IW_p(x, y) = \frac{\sqrt{(x - p_x)^2 + (y - p_y)^2}}{\maxdis(p_x, p_y)} \times (l_{min} - l_{max}) + l_{max} \quad (6)$$

where (p_x, p_y) indicates the coordinates of the simulated point light source, which is randomly selected in the whole image. $\maxdis(p_x, p_y)$ represent the maximum distance from (p_x, p_y) to the four corner of the image. l_{min} denotes the minimum illumination changing ratio, which is sampled uniformly from $[0.7, 0.9]$. And l_{max} denotes maximum illumination changing ratio, which is sampled uniformly from $[1.1, 1.3]$. (x, y) indicates the coordinates of the distribution weight matrix, and $x \in H, y \in W$.

As for line light source distortion, we generate the distribution weight matrix as:

$$IW_l \sim \mathbb{U}\{T^0, T^{90}, T^{180}, T^{270}\} \quad (7)$$

where

$$T^0(x, y) = \frac{(x - H) \times (l_{min} - l_{max})}{H} + l_{min} \quad (8)$$

T^{90}, T^{180}, T^{270} indicates the matrix of rotating T^0 with $90^\circ, 180^\circ, 270^\circ$ degrees respectively. IW_l is randomly chosen from $\{T^0, T^{90}, T^{180}, T^{270}\}$.

And the final illumination distortion ID is randomly chosen from $\{IW_p, IW_l\}$.

3.5.3 Moiré distortion. The most special distortion that occurs in screen-shooting process is the moiré distortion. Due to the sampling frequency difference between screen and camera, there might be several irregular textures appearing in the captured image. We simulate such distortion with the following equations according to

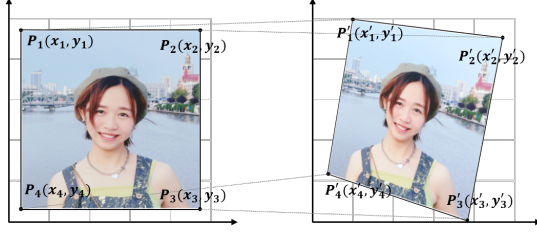


Figure 4: The perspective transformation.

[23].

$$\begin{aligned}
 Z_1(x, y) &= 0.5 + 0.5 \cos \left(2\pi \sqrt{(x - z_x)^2 + (y - z_y)^2} \right) \\
 Z_2(x, y) &= 0.5 + 0.5 \cos \left(\cos \left(\frac{\gamma}{\pi} \right) \times y + \sin \left(\frac{\gamma}{\pi} \right) \times x \right) \\
 Z(x, y) &= \min(Z_1(x, y), Z_2(x, y)) \\
 MD(x, y) &= (Z(x, y) + 1)/2
 \end{aligned} \quad (9)$$

where γ is selected uniformly from $[0, \pi]$, (z_x, z_y) represents the coordinates of one selected point, which is randomly sampled in the whole image. MD indicates the moiré distortion we simulated.

The final screen-shooting noise layer is generated by:

$$I_{no} = \delta_1 \times ID \times I_{PD} + \delta_2 \times MD + G_N \quad (10)$$

where G_N indicates the main part of the rest noise and it is simulated with Gaussian noise ($\sigma = 0.001$). δ_1 and δ_2 denote the corresponding ratio of distortion, which are set as 0.85 and 0.15 by default.

3.6 Decoder

The architecture of the proposed decoder is shown as Fig. 2. The decoder D aims to recover the watermark message from I_{no} . The whole decoder consists of five “single-conv” (conv-bn-relu-maxpool) blocks, nine “residual” blocks and one “linear” block, where the downsample operation is carried out three times in the “residual” blocks.

The objective of D is to minimize the difference between M_{de} and the original watermark M by updating θ_D :

$$\mathcal{L}_M = \text{MSE}(M, M_{de}) = \text{MSE}(M, D(\theta_D, I_{no})) \quad (11)$$

3.7 Gradient mask generator

In addition to training a strong decoder for better robustness, it is very important to generate better-watermarked image that is more conducive for decoding. To achieve so, we propose to use a gradient mask to guide the encoder. The gradient mask is generated by propagating back the message loss to the host image, which reflects the expected modifications of each pixel in order to correctly decode the watermark. We believe the region with a big gradient should be given priority to modification since it will lead to accurate decoding with a large weight.

The mask generation can be formulated as:

$$G_m = \frac{\partial \mathcal{L}_D}{\partial I_o} \quad (12)$$

Then we utilize the normalization form of G_m as the final gradient mask to guide the encoder. And the corresponding loss function can be written as:

$$\mathcal{L}_G = G'_m * \text{MSE}(I_o, I_{em}) = G'_m * \text{MSE}(I_o, E(\theta_E, I_o, M)) \quad (13)$$

where

$$G'_m = \left(1 - \frac{G_m - \min(G_m)}{\max(G_m) - \min(G_m)} \right) \times \alpha_G + \beta_G \quad (14)$$

And same as Eq. 2, α_G and β_G are used for adjusting the weights of the mask. In this paper, α_G and β_G are set as 1 and 1 by default.

3.8 Training Loss

The final loss of the whole network consists of the aforementioned edge mask-guided image loss, gradient mask-guided image loss, adversarial image loss and decoding loss, which can be formulated by:

$$\mathcal{L} = \lambda_1 \mathcal{L}_E + \lambda_2 \mathcal{L}_G + \lambda_3 \mathcal{L}_D + \lambda_4 \mathcal{L}_{Ad} \quad (15)$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are set as 0.5, 2, 3, 0.001 by default.

4 EXPERIMENTAL RESULTS

In this section, we will first introduce the implementation details of the proposed scheme. Then extensive experiments will be conducted to illustrate the performance of our method in view of visual quality and screen-shooting robustness. Finally, the ablation study will be carried out to justify our design.

4.1 Implementation Details

To train the whole network, we randomly choose 10000 images from the COCO dataset [17] as our training dataset. The whole framework is implemented by PyTorch [4] and executed on NVIDIA RTX 2080ti. All training images are resized to $128 \times 128 \times 3$ pixels. The length L of the random message M is set as 30. For parameter optimization, Adam [14] is applied with default hyperparameters. The network model is trained for 100 epochs with a batch size of 32. In the testing stage, all the test experiments are performed with the classical USC-SIPI image dataset [9]. The screen we use for experiments in Section 4.3.1-4.3.2 is “Lenovo LEN9053”, and the mobile phone is “Huawei P30 Pro”.

To measure the visual quality of the watermarked image, we utilize PSNR and SSIM as the default evaluation metrics. And for extracting accuracy and robustness evaluation, we directly use the extraction bit accuracy as the metric. We compared the proposed scheme with the state-of-the-art (SOTA) traditional watermarking method [6, 7] and three deep-learning-based method [18, 22, 24]. For LFM[24], we generate the training dataset of the CDTF network with “Lenovo LEN9053” and “Huawei P30 Pro”. And then we use the well-trained CDTF network as the noise layer to train the whole framework.

4.2 Visual quality

In this section, we will compare the visual quality of the watermarked image with different schemes. The results are shown in Fig. 5 and Table 1.

We can see from Table 1 that the PSNR value of all the schemes ranges from 29 dB to 36 dB. Since the embedding intensity of traditional watermarking methods [6, 7] can be adjusted, we set them at the same level as the proposed method for fair comparison. As for the deep-learning-based method [18, 22, 24], we utilize watermarked images embedded with the most robust pre-trained epoch for experiments, and we can see that the PSNR and SSIM values are

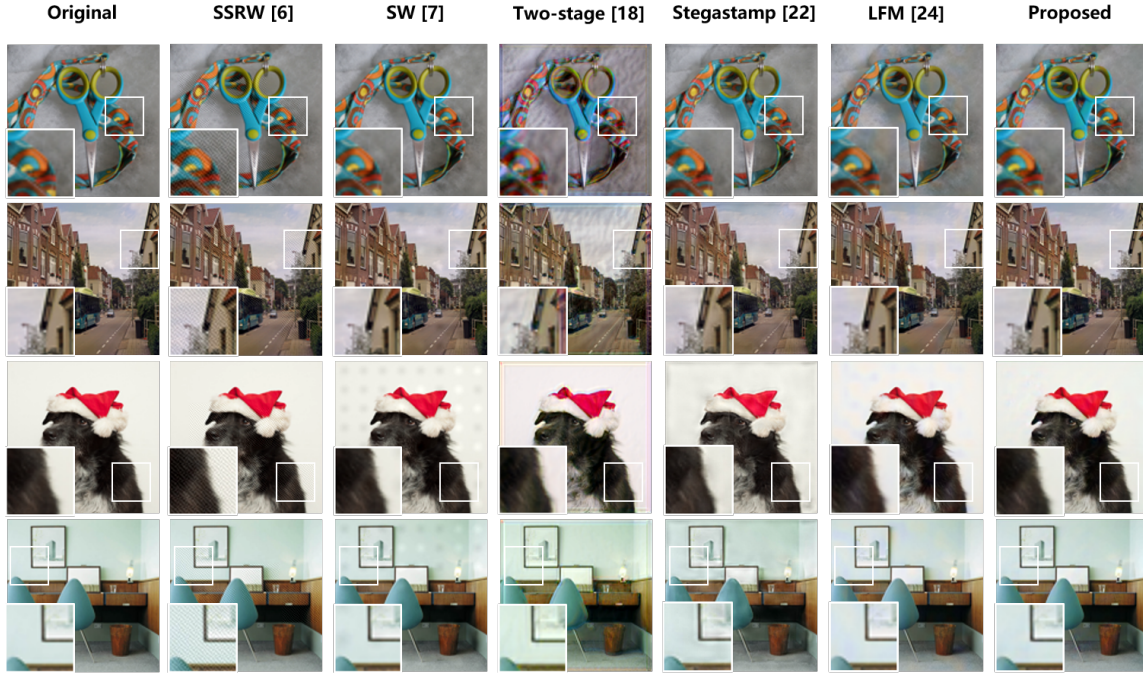


Figure 5: The visual quality of watermarked images embedded with different methods.

Table 1: The PSNR and SSIM values of each methods.

Methods	SSRW [6]	SW [7]	Two-stage [18]
PSNR(dB)	36.18	36.28	30.35
SSIM	0.985	0.986	0.907
Methods	Stegastamp [22]	LFM [24]	Proposed
PSNR(dB)	29.88	33.92	36.21
SSIM	0.930	0.975	0.985

Table 2: Extraction accuracy with different shooting distance.

Distance(cm)	20	30	40	50	60
SSRW [6]	97.35%	97.88%	95.75%	87.81%	91.87%
SW [7]	77.44%	72.76%	67.78%	72.95%	69.04%
Two-stage [18]	53.71%	54.23%	52.33%	54.32%	51.01%
Stegastamp [22]	95.24%	92.50%	94.00%	95.25%	96.26%
LFM [24]	92.92%	92.71%	91.46%	92.50%	91.04%
Proposed	99.38%	98.75%	97.92%	97.29%	98.75%

lower than the proposed scheme. Besides, from Fig. 5 we can see that even the PSNR and SSIM are in the same value, the visual quality is quite different with each scheme. From the locally enlarged image block we can see that [6] and [7] will produce serious visual distortion in embedding regions, but our scheme will maintain high visual quality.

4.3 Screen-shooting Robustness Test

In this section, we will show and discuss the screen-shooting robustness of the proposed scheme. Specifically, we embed randomly sampled messages in all the images in dataset [9] with different schemes. And we further display the watermarked images on the screen, and use mobile phones to capture them in different conditions. For captured images, we first detect and perspective correct the watermarked image in each photo and then use the decoder to extract the watermark in the corrected images, the detection and correction process of the watermarked images are same as the settings in [22].

4.3.1 Distance Test. In order to test the robustness in the view of shooting distance, we conduct the corresponding experiments. We display the watermarked image on the screen and use mobile phone

to capture them at different distances. The shooting distance ranges from 20cm to 60cm. And the corresponding extraction results are shown in Table 2.

From Table 2 we can see that in all the shooting distance, the proposed scheme maintains a high-level extraction accuracy which is higher than 97%. And for all the distances, the proposed scheme maintains the best extraction performance. Compared with LFM [24], the extraction accuracy of the proposed scheme is at least 4% higher, which greatly indicates the effectiveness of the screen-shooting noise layer. And the results in Table 2 indicate the strong robustness of the proposed scheme in screen-shooting process in the view of shooting distances. But we can find that the accuracy does not decrease with the increase of shooting distance, we conclude the reason as follows: Shooting at a large distance may influence the recording of the image details which will behave as less proportion of image pixels in the final photo. But the exiting pixel is enough to realize the watermark extraction, so the increasing distance will not certainly result in the decrease of extraction accuracy. Nevertheless, it should be noted that when the distance is too far, the specific details of the watermarked images may not be successfully recorded,

Table 3: Extraction accuracy with different shooting angles.

Angles(°)	Left 40/Up 40	Left 30/Up 30	Left 20/Up 20	Left 10/Up 10	Right 10/Down 10	Right 20/Down 20	Right 30/Down 30	Right 40/Down 40
SSRW [6]	88.56%/94.38%	89.25%/92.81%	88.94%/87.69%	90.87%/89.50%	92.25%/93.75%	92.69%/89.25%	92.56%/85.94%	85.62%/93.69%
SW [7]	64.16%/66.31%	68.36%/66.60%	71.78%/68.55%	75.49%/73.93%	76.37%/75.49%	69.14%/70.70%	68.95%/70.21%	65.82%/67.68%
Two-stage [18]	52.50%/54.17%	56.67%/51.16%	53.23%/53.33%	53.34%/56.67%	52.52%/52.50%	53.43%/55.00%	56.66%/53.32%	55.83%/50.83%
Stegastamp [22]	88.00%/96.75%	95.75%/96.25%	96.25%/95.00%	96.25%/96.00%	94.75%/95.75%	96.75%/96.00%	96.75%/95.25%	96.00%/95.75%
LFM [24]	84.79%/91.46%	89.17%/89.58%	91.04%/90.21%	88.54%/90.42%	89.17%/91.46%	89.38%/91.45%	90.00%/91.67%	90.63%/91.04%
Proposed	97.08%/98.75%	98.13%/98.75%	98.75%/99.17%	98.75%/98.96%	98.96%/99.16%	98.95%/98.96%	98.96%/98.75%	99.38%/99.17%

so the extraction accuracy of photos with long-distance capturing may be low.

4.3.2 Angle Test. To test the screen-shooting robustness in different horizontal and vertical shooting angles, we display the watermarked image on the screen and use mobile phone to capture them at different angles, the shooting distance is fixed at 50cm. The angle ranges from left 40° to right 40° and up 40° to down 40° with the intervals of 10°. And the corresponding extraction results are shown in Table 3.

Table 3 indicates the robustness with different shooting angles. It can be seen that in all the conditions, the proposed scheme maintains the best extraction accuracy compared with other schemes. The corresponding extraction accuracy of the proposed scheme is larger than 97% in all horizontal angles and larger than 98% in all the vertical shooting angles, which means that such network can well satisfy the shooting angles from -40° to 40°. Such results illustrate that the algorithm can be applied in most cases in actual scenarios, because screen-shooting at more than 40° is too narrow to well record the contents on the screen. Besides, we can see that compared with LFM [24], the extraction performance is at least 7% higher, which also reveals that the screen-shooting noise layer we designed can effectively help the network learn better features and make the whole system to be robust against screen-shooting distortions.

4.3.3 Device Test. The aforementioned experiments are carried out under the device of “LEN9053” and “Huawei P30 Pro”, but whether the proposed method is useful in other devices is also important for practical scenarios. To test the adaptability of the proposed scheme, we conduct screen-shooting experiments with different types of screens and mobile phones. The shooting distance is fixed at 20cm. And the test results are shown in Table 4.

Table 4: Extraction accuracy with different devices compared with LFM [24].

Device		P30 Pro	iPhone 12	iPhone SE	RedMi Note 10
LEN9053	LFM [24]	92.92%	92.28%	91.15%	92.13%
	Proposed	99.38%	99.38%	99.17%	97.29%
Retina	LFM [24]	91.04%	90.00%	91.25%	92.15%
	Proposed	98.54%	99.17%	99.17%	98.13%
LEN40A9	LFM [24]	91.75%	92.26%	91.15%	92.08%
	Proposed	99.17%	98.96%	99.16%	97.08%

We use three different screens (“Lenovo LEN9053”, “Macbook Air Retina” and “Lenovo LEN40A9”) and four different mobile phones (“Huawei P30 Pro”, “iPhone 12”, “iPhone SE” and “Redmi Note 10”)

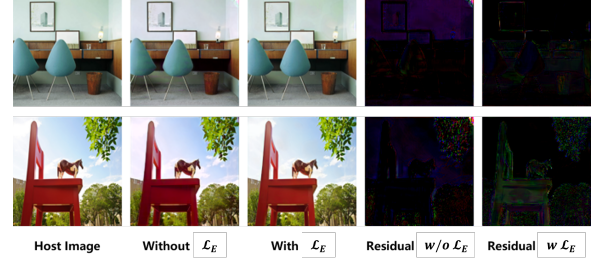


Figure 6: The watermarked image with/without edge mask-guided loss. Residual w/o $\mathcal{L}_E$ indicates the residual image generated by the network training without edge mask-guided loss. Residual w $\mathcal{L}_E$ indicates the residual image generated by the network training with edge mask-guided loss.

for experiments. From Table 4 we can see that for all the device pairs, the proposed scheme can achieve high-level extraction accuracy, which is higher than 97%. But for LFM [24], the extraction accuracy is around 91% to 92%, which is at least 6% lower than the proposed scheme. This indicates the great adaptability of the proposed scheme. We conclude the strong adaptability of the proposed scheme comes from the noise layer design, because no matter what types of devices are utilized, the most influenced distortions are similar. So once we can train the network to be adapted to these distortions, the robustness of different devices can be effectively guaranteed.

Table 5: Extraction accuracy with different shooting distance w&w/o $\mathcal{L}_G$.

Distance(cm)	20	30	40	50	60
w/o $\mathcal{L}_G$	98.96%	98.54%	96.16%	96.17%	96.87%
w $\mathcal{L}_G$	99.38%	98.75%	97.92%	97.29%	98.75%

4.4 Ablation Study

Since we propose two mask-guided losses for better performance, in this section, we conduct the ablation study to justify our design of these two losses.

4.4.1 The importance of $\mathcal{L}_G$. We propose a gradient mask-guided image loss for better robustness in this paper. To justify this design, we first train two models with and without $\mathcal{L}_G$ respectively. And then we use these two networks with epoch-100 to perform the screen-shooting experiments as illustrated in Section 4.3.1 and 4.3.2. The corresponding results are shown in Table 5. To simplify the express, we use w/o $\mathcal{L}_G$ to denote the network trained without gradient mask-guided image loss, and w $\mathcal{L}_G$ to denote the network trained with gradient mask-guided image loss.

As for the robustness, we use the extraction accuracy with different shooting distances and different shooting angles for judgment.

Table 6: Extraction accuracy with different shooting angles w&w/o $\mathcal{L}_G$.

Angles(°)	Left 40/Up 40	Left 30/Up 30	Left 20/Up 20	Left 10/Up 10	Right 10/Down 10	Right 20/Down 20	Right 30/Down 30	Right 40/Down 40
w/o $\mathcal{L}_G$	93.46%/96.46%	96.08%/95.08%	97.71%/96.70%	97.79%/96.79%	95.79%/95.80%	97.12%/96.12%	97.00%/95.67%	96.95%/95.95%
w $\mathcal{L}_G$	97.08%/98.75%	98.13%/98.75%	98.75%/99.17%	98.75%/98.96%	98.96%/99.16%	98.95%/98.96%	98.96%/98.75%	99.38%/99.17%

Table 5-6 show that for all the shooting conditions, training with $\mathcal{L}_G$ gets better performance than training without $\mathcal{L}_G$, which reveals the importance of gradient mask-guided image loss. With $\mathcal{L}_G$, the encoder can create better-watermarked images which are easier to decode by the decoder. Besides, comparing the results with Table 3 and Table 2, even if training without $\mathcal{L}_G$, the performance of the proposed scheme still outperforms the compared schemes, which indicates that for the framework proposed in this paper, the main performance advantage comes from the design of noise layer. $\mathcal{L}_G$ gives around 1% improvement for the robustness.

4.4.2 The importance of $\mathcal{L}_E$. As aforementioned, we design $\mathcal{L}_E$ to guide the watermark embedding for better visual quality. To justify this design, we train two models with and without edge mask-guided image loss respectively. Specifically, we output the watermarked images with the same iteration epoch to show the importance of $\mathcal{L}_E$. The corresponding result is shown in Fig. 6.

As seen in Fig. 6, training with edge mask-guided image loss will make the modification concentrate more on the edge region. Besides, there are two main advantages that training with edge mask-guided image loss: 1) It will inhibit the generation of chromatic aberration. 2) It will inhibit the generation of abnormal textures.

As shown in Fig. 6, training without edge mask-guided image loss will create chromatic aberration and some abnormal textures in the top-right of the image, but this will not happen in training with $\mathcal{L}_E$. The reason can be concluded that edge mask-guided image loss gives the edge region smaller weights in calculating the image loss, which means it encourages the modifications of the edge region. So with $\mathcal{L}_E$, the modification on other regions will be discouraged. But only applying MSE loss without $\mathcal{L}_E$ will give the same weights on each pixel, which could lead to the result that the network may modify the smooth region of the image to embed the watermark, and such modification may produce chromatic aberration and abnormal textures. So in all, training with $\mathcal{L}_E$ can effectively make the modification concentrating more on edge region. But it is worth noting that, we found when training a large epoch without $\mathcal{L}_E$, the visual quality will also improve, but with $\mathcal{L}_E$, the needed iterative epoch will be significantly lessened.

4.5 Limitations

Although the proposed framework can work with a high rate of extraction accuracy against screen-shooting process, there are still some limitations when broad deployment occurred. One is that the watermarked image must be fully captured, otherwise, it cannot be perspective corrected and further being decoded. Future work could improve upon our architecture to acquire the extraction ability with partial image recorded. Additionally, the decoding performance is influenced by the precision detection and perspective correction. A better detecting network will improve the extraction accuracy of the whole system.

5 APPLICATIONS

In this section, we will show three typical applications of the proposed PIMoG noise layer, which further demonstrate the broad applicability of our method.

1) Screen-shooting Resilient Watermarking. PIMoG noise layer can effectively guide the design of watermark algorithms for screen-shooting robustness. No matter the algorithm is based on deep-learning or handcrafted image processing operation, PIMoG can serve as a distortion analyzer to help the algorithm design.

2) Screen-to-camera Image Code. Screen-to-camera image code can be used as a way to realize screen-to-camera message transmission. PIMoG can help to design better codec in order to resist the screen-shooting distortions.

3) Adversarial Attack and Defense. Machine learning models are vulnerable to adversarial attacks, a stronger attack can help to generate a better defense system. PIMoG can be used for generating the screen-shooting robust adversarial examples, which could further help the defense of adversarial attacks in real scenes.

6 CONCLUSION

In this paper, we give new insight in designing the screen-shooting noise layer for deep-learning-based watermarking framework, that is: it is not necessary to simulate the overall distortions of the screen-shooting process as a noise layer, only including the most influenced distortions in the noise layer is enough to guarantee the corresponding robustness. This may inspire the follow-up noise layer design when facing complex distortions. As for the details, we conclude the most influenced distortions in screen-shooting process into three main parts: perspective distortion, illumination distortion and moiré distortion. And we further simulate these distortions in a differentiable way and generate the screen-shooting noise layer dubbed PIMoG, with which, the watermarking network can be end-to-end trained and the robustness against screen-shooting distortion can be effectively guaranteed. Besides, to improve such robustness, we also propose a gradient mask-guided image loss in training stage, which helps to guide the encoder to generate easier-decoding features. As for invisibility, we propose an edge mask-guided image loss to better constrain the embedding region. With such loss, the watermarked images can maintain high visual quality. Extensive experiments indicate the great performance of the designed noise layer and the strong robustness of the proposed scheme in different screen-shooting conditions.

ACKNOWLEDGEMENT

This research is supported by the National Research Foundation, Singapore under its Strategic Capability Research Centres Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

REFERENCES

- [1] Mehran Andalibi and Damon M. Chandler. 2015. Digital Image Watermarking via Adaptive Logo Texturization. *IEEE Trans. Image Process.* 24, 12 (2015), 5060–5073.
- [2] Shumeet Baluja. 2019. Hiding images within images. *IEEE transactions on pattern analysis and machine intelligence* 42, 7 (2019), 1685–1697.
- [3] Chia-Sung Chang and Jau-Ji Shen. 2017. Features Classification Forest: A Novel Development that is Adaptable to Robust Blind Watermarking Techniques. *IEEE Trans. Image Process.* 26, 8 (2017), 3921–3935.
- [4] Ronan Collobert, Koray Kavukcuoglu, and Clement Farabet. 2011. Torch7: A Matlab-like Environment for Machine Learning. (2011).
- [5] Han Fang, Dongdong Chen, Qidong Huang, Jie Zhang, Zehua Ma, Weiming Zhang, and Nenghai Yu. 2020. Deep Template-based Watermarking. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 4 (2020), 1436–1451.
- [6] Han Fang, Weiming Zhang, Hang Zhou, Hao Cui, and Nenghai Yu. 2018. Screen-Shooting Resilient Watermarking. *IEEE Transactions on Information Forensics and Security* 14, 6 (2018), 1403–1418.
- [7] David Gugelmann, David Sommer, Vincent Lenders, Markus Happe, and Laurent Vanbever. 2018. Screen watermarking for data theft investigation and attribution. In *2018 10th International Conference on Cyber Conflict (CyCon)*. IEEE, 391–408.
- [8] Jianzhong He, Shiliang Zhang, Ming Yang, Yanhu Shan, and Tiejun Huang. 2020. Bdcn: Bi-directional cascade network for perceptual edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020).
- [9] Available: <http://sipi.usc.edu/database/>. 2022. The USC-SIPI Image Database. Accessed: Mar. 2022 [Online]. (2022).
- [10] Phillip Isola, Junyan Zhu, Tinghui Zhou, and Alexei A Efros. 2017. Image-to-Image Translation with Conditional Adversarial Networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 5967–5976.
- [11] Zhaoyang Jia, Han Fang, and Weiming Zhang. 2021. Mbrs: Enhancing robustness of dnn-based watermarking by mini-batch of real and simulated jpeg compression. In *Proceedings of the 29th ACM International Conference on Multimedia*. 41–49.
- [12] Xiangui Kang, Jiwu Huang, and Wenjun Zeng. 2010. Efficient general print-scanning resilient data hiding based on uniform log-polar mapping. *IEEE Trans. Inf. Forensics Secur.* 5, 1 (2010), 1–12.
- [13] Xiangui Kang, Rui Yang, and Jiwu Huang. 2011. Geometric Invariant Audio Watermarking Based on an LCM Feature. *IEEE Trans. Multimedia* 13, 2 (2011), 181–190.
- [14] Diederik P Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*.
- [15] Chen-Tsung Kuo and Shyi-Chyi Cheng. 2007. Fusion of color edge detection and color quantization for color image watermarking using principal axes analysis. *Pattern Recognition* 40, 12 (2007), 3691–3704.
- [16] Gerrit C Langelaar and Reginald L Lagendijk. 2001. Optimal differential energy watermarking of DCT encoded images and video. *IEEE Transactions on image Processing* 10, 1 (2001), 148–158.
- [17] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *European conference on computer vision*. Springer, 740–755.
- [18] Yang Liu, Mengxi Guo, Jian Zhang, Yuesheng Zhu, and Xiaodong Xie. 2019. A Novel Two-stage Separable Deep Learning Framework for Practical Blind Watermarking. In *Proceedings of the 27th ACM International Conference on Multimedia, MM 2019, Nice, France, October 21-25, 2019*. ACM, 1509–1517.
- [19] Benjamin Mathon, François Cayre, Patrick Bas, and Benoît Macq. 2014. Optimal Transport for Secure Spread-Spectrum Watermarking of Still Images. *IEEE Trans. Image Process.* 23, 4 (2014), 1694–1705.
- [20] Takao Nakamura, Atsushi Katayama, Masashi Yamamuro, and Noboru Sonehara. 2004. Fast watermark detection scheme for camera-equipped cellular phone. In *Proceedings of the 3rd international conference on Mobile and ubiquitous multimedia*. ACM, 101–108.
- [21] Anu Pramila, Anja Keskinarkaus, Valtteri Takala, and Tapio Seppänen. 2017. Extracting watermarks from printouts captured with wide angles using computational photography. *Multimedia Tools and Applications* 76, 15 (2017), 16063–16084.
- [22] Matthew Tancik, Ben Mildenhall, and Ren Ng. 2019. StegaStamp: Invisible Hyperlinks in Physical Photographs. *arXiv preprint arXiv:1904.05343* (2019).
- [23] Pei-Hen Tsai and Yung-Yu Chuang. 2013. Target-driven moire pattern synthesis by phase modulation. In *Proceedings of the IEEE International Conference on Computer Vision*. 1912–1919.
- [24] Eric Wengrowski and Kristin Dana. 2019. Light Field Messaging With Deep Photographic Steganography. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 1515–1524.
- [25] Chaoning Zhang, Philipp Benz, Adil Karjauv, Geng Sun, and In So Kweon. 2020. Udh: Universal deep hiding for steganography, watermarking, and light field messaging. *Advances in Neural Information Processing Systems* 33 (2020), 10223–10234.
- [26] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph Stoecklin, Heqing Huang, and Ian Molloy. 2018. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*. 159–172.
- [27] Peiying Zhang, Chenhui Li, and Changbo Wang. 2020. Viscode: Embedding information in visualization images using encoder-decoder network. *IEEE Transactions on Visualization and Computer Graphics* 27, 2 (2020), 326–336.
- [28] Jiren Zhu, Russell Kaplan, Justin Johnson, and Li Fei-Fei. 2018. Hidden: Hiding data with deep networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 657–672.
- [29] Tianrui Zong, Yong Xiang, Iynkaran Natgunanathan, Song Guo, Wanlei Zhou, and Gleb Beliakov. 2015. Robust Histogram Shape-Based Method for Image Watermarking. *IEEE Trans. Circuits Syst. Video Technol.* 25, 5 (2015), 717–729.